# Azure Service Bus – Reading a Message and Deserializing JSON into a C# Object

## 1. Objective

In this example, we will:

1. Read a message from an **Azure Service Bus Queue**.
2. Convert the message body into a JSON string.
3. Deserialize the JSON string into a custom C# `Employee` object.
4. Access the employee properties such as `FirstName`, `LastName`, and `Salary`.

### Overall Flow

**Azure Service Bus Queue → Message → JSON String → C# Employee Object**

---

# 2. Example Message in Azure Service Bus

Suppose the message stored in the Azure Service Bus Queue is:

```
{
    "FirstName": "Henry",
    "LastName": "Jane",
    "Salary": 25000
}
```

This message is stored as JSON.

Our goal is to convert this JSON into an `Employee` object.

---

# 3. Employee Class

First, we create a custom C# class.

```
public class Employee
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
```

```
    public decimal Salary { get; set; }
}
```

The JSON properties should match the properties of the C# class.

For example:

| JSON Property | C# Property |
|---|---|
| FirstName | FirstName |
| LastName | LastName |
| Salary | Salary |

After deserialization, the values from the JSON will be assigned to the corresponding C# properties.

---

# 4. Required NuGet Packages

We need the Azure Service Bus package to communicate with Azure Service Bus.

### Azure.Messaging.ServiceBus

This package provides classes such as:

- `ServiceBusClient`
- `ServiceBusReceiver`
- `ServiceBusMessage`
- `ServiceBusProcessor`

We also need **Newtonsoft.Json** for JSON deserialization.

### Newtonsoft.Json

It provides:

```
JsonConvert.DeserializeObject<T>()
```

which can convert a JSON string into a C# object.

---

# 5. Connecting to Azure Service Bus

First, we need the Service Bus connection string and queue name.

Example:

```
string connectionString = "YOUR_CONNECTION_STRING";
string queueName = "employeequeue";
```

Then create a `ServiceBusClient`.

```
ServiceBusClient client = new ServiceBusClient(connectionString);
```

### What is ServiceBusClient?

`ServiceBusClient` is used to establish communication with Azure Service Bus.

Think of it as the **connection/client used to communicate with the Service Bus namespace**.

---

# 6. Creating a Receiver

Next, create a receiver for the queue.

```
ServiceBusReceiver receiver =
    client.CreateReceiver(queueName);
```

The receiver is responsible for receiving messages from the specified queue.

### Simple Understanding

```
ServiceBusClient
       ↓
ServiceBusReceiver
       ↓
Azure Service Bus Queue
       ↓
Message
```

---

# 7. Reading the Message

We can receive a message using:

```
ServiceBusReceivedMessage message =
    await receiver.ReceiveMessageAsync();
```

The returned `message` contains information about the Service Bus message.

The actual application data is available in the message body.

---

# 8. Converting Message Body to String

The message body can be converted into a string:

```
string messageBody = message.Body.ToString();
```

Now `messageBody` contains the JSON string.

For example:

```
{"FirstName":"Henry","LastName":"Jane","Salary":25000}
```

At this point:

```
Azure Service Bus Message
          ↓
message.Body
          ↓
JSON String
```

---

# 9. Deserialization

Now we need to convert the JSON string into our custom `Employee` object.

We use:

```
Employee employee1 =
    JsonConvert.DeserializeObject<Employee>(messageBody);
```

### What does this mean?

Let's break it down:

```
JsonConvert.DeserializeObject<Employee>(messageBody);
```

`JsonConvert`

Comes from the **Newtonsoft.Json** package.

`DeserializeObject<Employee>`

Means:

> Convert the JSON data into an object of type `Employee`.

`messageBody`

This is the JSON string that we received from Azure Service Bus.

---

# 10. What Happens During Deserialization?

Suppose the Service Bus contains:

```
{
    "FirstName": "Henry",
    "LastName": "Jane",
    "Salary": 25000
}
```

And our class is:

```
public class Employee
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
```

```
    public decimal Salary { get; set; }
}
```

When we execute:

```
Employee employee1 =
    JsonConvert.DeserializeObject<Employee>(messageBody);
```

Newtonsoft.Json maps the values.

The result will be:

```
employee1.FirstName = "Henry"
employee1.LastName  = "Jane"
employee1.Salary    = 25000
```

So the JSON has been converted into a strongly typed C# object.

---

# 11. Complete Example

```
using Azure.Messaging.ServiceBus;
using Newtonsoft.Json;

class Employee
{
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public decimal Salary { get; set; }
}

class Program
{
    static async Task Main(string[] args)
    {
        string connectionString = "YOUR_CONNECTION_STRING";
        string queueName = "employeequeue";

        ServiceBusClient client =
            new ServiceBusClient(connectionString);

        ServiceBusReceiver receiver =
            client.CreateReceiver(queueName);
```

```
        ServiceBusReceivedMessage message =
            await receiver.ReceiveMessageAsync();

        string messageBody = message.Body.ToString();

        Employee employee1 =
            JsonConvert.DeserializeObject<Employee>(messageBody);

        Console.WriteLine(employee1.FirstName);
        Console.WriteLine(employee1.LastName);
        Console.WriteLine(employee1.Salary);
    }
}
```

---

# 12. Important Concept – Serialization vs Deserialization

This is an important interview concept.

## Serialization

**C# Object → JSON**

Example:

```
Employee Object
      ↓
Serialization
      ↓
JSON
```

Example:

```
string json = JsonConvert.SerializeObject(employee);
```

---

## Deserialization

**JSON → C# Object**

Example:

```
JSON
 ↓
Deserialization
 ↓
Employee Object
```

Example:

```
Employee employee =
    JsonConvert.DeserializeObject<Employee>(json);
```

### Easy Way to Remember

**Serialization:** Object → JSON

**Deserialization:** JSON → Object

---

# 13. Why Do We Deserialize the Message?

Azure Service Bus does not automatically know that our message represents an `Employee` object.

The message is basically data.

For example:

```
{
    "FirstName": "Henry",
    "LastName": "Jane",
    "Salary": 25000
}
```

Our application needs to convert this data into a meaningful C# type:

```
Employee employee
```

This makes it easier to work with the data.

Instead of doing:

```
// Working with raw JSON
```

we can do:

```
employee.FirstName
employee.LastName
employee.Salary
```

This is much easier and type-safe.

---

# 14. Important Azure Service Bus Classes

### ServiceBusClient

Used to connect to Azure Service Bus.

```
ServiceBusClient client;
```

### ServiceBusReceiver

Used to receive messages from a queue or subscription.

```
ServiceBusReceiver receiver;
```

### ServiceBusReceivedMessage

Represents a message received from Service Bus.

```
ServiceBusReceivedMessage message;
```

### Message Body

Contains the actual application data.

```
message.Body
```

---

# 15. Complete Flow to Remember

For interviews, remember this flow:

```
Azure Service Bus
        ↓
     Queue
        ↓
ServiceBusReceiver
        ↓
ReceiveMessageAsync()
        ↓
ServiceBusReceivedMessage
        ↓
message.Body
        ↓
JSON String
        ↓
JsonConvert.DeserializeObject<Employee>()
        ↓
Employee Object
        ↓
employee.FirstName
employee.LastName
employee.Salary
```

---

# 16. Real-Time Scenario

Consider an **Employee Service**.

Another application wants to send employee information to our application.

The producer sends:

```
{
    "FirstName": "Henry",
    "LastName": "Jane",
    "Salary": 25000
}
```

The message is placed in an Azure Service Bus Queue.

Our application receives the message:

```
var message =
    await receiver.ReceiveMessageAsync();
```

Then gets the JSON:

```
string json = message.Body.ToString();
```

Then converts it into an object:

```
Employee employee =
    JsonConvert.DeserializeObject<Employee>(json);
```

Now our application can use:

```
employee.FirstName
employee.LastName
employee.Salary
```

For example:

```
Console.WriteLine(
    $"Employee: {employee.FirstName} {employee.LastName}");

Console.WriteLine(
    $"Salary: {employee.Salary}");
```

---

# 17. Interview Questions

## Q1. What is deserialization?

Deserialization is the process of converting data such as JSON into a C# object.

```
JSON → C# Object
```

---

## Q2. Which package is used in this example for JSON deserialization?

**Newtonsoft.Json**

The main method is:

```
JsonConvert.DeserializeObject<T>()
```

---

## Q3. What is the difference between serialization and deserialization?

| Serialization | Deserialization |
|---|---|
| Object → JSON | JSON → Object |
| Used when sending/storing object data | Used when reading JSON data |
| `SerializeObject()` | `DeserializeObject<T>()` |

---

## Q4. What is `ServiceBusClient`?

`ServiceBusClient` is used to establish communication with Azure Service Bus.

---

## Q5. What is `ServiceBusReceiver`?

`ServiceBusReceiver` is used to receive messages from an Azure Service Bus queue or subscription.

---

## Q6. How do you read a message from Azure Service Bus?

```
var message =
    await receiver.ReceiveMessageAsync();
```

---

### Q7. How do you get the message body?

```
string messageBody = message.Body.ToString();
```

---

### Q8. How do you convert the JSON message into an Employee object?

```
Employee employee =
    JsonConvert.DeserializeObject<Employee>(messageBody);
```

---

# 18. One-Minute Interview Explanation

If the interviewer asks:

**"How do you read and process a JSON message from Azure Service Bus?"**

You can answer:

> First, I create a `ServiceBusClient` using the Service Bus connection string. Then I create a `ServiceBusReceiver` for the required queue and receive the message using `ReceiveMessageAsync()`. I extract the message body and convert it into a JSON string. Then I deserialize the JSON using `JsonConvert.DeserializeObject<Employee>()`, which maps the JSON properties to the corresponding properties of my `Employee` class. Finally, I can use the strongly typed Employee object in my application.

---

# 19. Key Points to Remember

- Azure Service Bus Queue stores and delivers messages.
- `ServiceBusClient` is used to connect to Service Bus.
- `ServiceBusReceiver` receives messages.
- `ReceiveMessageAsync()` reads a message.
- `message.Body` contains the message payload.
- The payload can be JSON.
- `JsonConvert.DeserializeObject<T>()` converts JSON into a C# object.
- **Serialization:** Object → JSON.
- **Deserialization:** JSON → Object.

- Property names in the JSON should generally match the C# model properties for straightforward mapping.

## Quick Revision

```
Queue
 ↓
Receive Message
 ↓
message.Body
 ↓
JSON String
 ↓
Deserialize
 ↓
Employee Object
 ↓
Use Employee Properties
```

**Most important line:**

```
Employee employee =
    JsonConvert.DeserializeObject<Employee>(messageBody);
```

This single line converts the JSON message received from Azure Service Bus into our custom `Employee` C# object.